

AUDIT TECHNIQUE, PRODUIT ET UX

TOK vers niveau Uber Eats

Audit complet du code, des logiques backend, de Supabase, Stripe, GitHub, frontend et UX, avec roadmap de correction priorisee.

Repo: mtnrconcept/cloud-rebuild

Workspace: C:\Users\Pc\cloud-rebuild-recovered

Branche: main

Date: 16 juin 2026

Decision: ne pas pousser production tant que les P0 ne sont pas corriges et verifies

Date: 16 juin 2026

Repo: [mtnrconcept/cloud-rebuild](#)

Workspace: [C:\Users\Pc\cloud-rebuild-recovered](#)

Branche auditee: [main](#)

Objectif: audit complet code, logique, fonctionnalites, frontend, UX, Supabase, GitHub, readiness et ecart avec une experience type Uber Eats.

Verdict executif

TOK a deja une base produit tres large: app client, dashboard restaurateur, courier app, admin back-office, feature flags, Supabase/RLS, Stripe, notifications, social feed, reservations, anti-gaspi, ventes flash, Tok One, operations center et logique mobile. Les validations locales principales passent: `pnpm lint`, `pnpm lint:release`, `pnpm test` avec 891 tests, `pnpm run build`, `pnpm run build:prod`, et `pnpm exec tsc --noEmit`.

Mais l'application n'est pas encore au niveau de confiance operationnelle d'Uber Eats. Les blocages sont concentres sur quatre zones: idempotence Stripe, surface publique Supabase d'onboarding, verification live Supabase/Auth, et alignement environnements/migrations. Cote UX, le mobile est proche d'un parcours actionnable apres l'intro, mais l'intro plein ecran et le desktop trop hero-first retardent l'action principale: chercher et commander/reserver.

Decision: ne pas pousser production tant que les P0 ci-dessous ne sont pas fermes et verifies par tests, readiness scripts et verification live Supabase.

Perimetre audite

- 651 fichiers TypeScript/TSX sous `src`.
- 72 fichiers TypeScript d'Edge Functions Supabase.
- 270 migrations SQL Supabase.
- 242 fichiers de tests.
- Workflows GitHub Actions CI et production.
- `package.json`, `vercel.json`, routage React, auth, feature flags, checkout, webhook Stripe, onboarding, RLS/migrations, fonctions publiques, UI publique et protections de routes.
- GitHub: issue ouverte #204 et 5 PR ouvertes.
- Supabase: verification locale via scripts du repo et tentative connecteur Supabase.
- Navigateur: verification locale <http://127.0.0.1:5173/> avec Browser integre puis Playwright/Chrome local pour captures fiables.

Limites: pas de credentials utilisateur fournis, donc pas de validation visuelle authentifiee admin/restaurateur/coursier. Le connecteur Supabase n'a pas acces au projet TOK `wcrtyoueexykkikaos`; il liste seulement un autre projet. Les tests live de paiement Stripe et production Supabase n'ont pas ete executes.

Benchmark Uber Eats utilise

Sources officielles consultees:

- Uber Eats Marketplace APIs: gestion programmatique des stores, menus, commandes, promotions et operations multi-sites. Source: <https://developer.uber.com/docs/eats/introduction>
- Uber Eats Manager roles: Admin, Manager, Staff avec acces differencies aux menus, analytics, paiements, taxes, utilisateurs et marketing. Source: <https://help.uber.com/merchants-and-restaurants/article/managing-access-in-uber-eats-manager/?nodeId=931d0f36-f871-4c8a-be24-ca22ef9b45ff>
- Uber Eats Manager analytics: ventes, volume de commandes, ticket moyen, online rate, issues, heatmaps, top items, menu feedback. Source: <https://help.uber.com/en/merchants-and-restaurants/article/how-can-i-view-my-performance-data-in-uber-eats-manager?nodeId=980a4325-3a80-42d5-83ae-2876982a5994>
- Scheduled orders: notification plein ecran, section commandes planifiees, apparition en haut de file au moment de preparation. Source: <https://help.uber.com/en/merchants-and-restaurants/article/what-are-scheduled-orders?nodeId=858cd1be-ce17-4a3b-a819-b941c2114b63>
- Group orders: panier partage, limite de depense, deadline, paiement centralise ou separe, tracking commun. Source: <https://www.uber.com/au/en/blog/split-your-uber-eats-order-with-the-team/>

- Courier experience: earnings upfront, tips, navigation, support in-app, Instant Pay jusqu'a 6 cashouts par jour. Source: <https://www.uber.com/us/en/deliver/>
- Supabase password security: leaked password protection via HavelBeenPwned, disponible Pro+. Source: <https://supabase.com/docs/guides/auth/password-security>

Ce qui est solide aujourd'hui

- Les routes critiques sont protegees par `ProtectedRoute`, `DashboardRoute` et `AdminProtectedRoute`; les routes admin redirigent bien vers `/auth?redirect=...` sans session.
- Les feature flags sont integres dans le routage et les surfaces publiques.
- Les validations locales principales passent:
 - `pnpm run lint`: OK.
 - `pnpm run lint:release`: OK.
 - `pnpm run test`: 241 test files, 891 tests, OK.
 - `pnpm run build`: OK.
 - `pnpm run build:prod`: OK, SEO prerender 73 routes publiques.
 - `pnpm exec tsc --noEmit`: OK.
- `vercel.json` contient des headers de securite utiles: `CSP`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`, `X-Frame-Options`.
- Les Edge Functions utilisent largement `authenticateRequest`, `requireUserRole`, rate limits, secrets scheduler, signatures Stripe ou HMAC selon les cas.
- Les migrations montrent un effort RLS serieux, avec `SECURITY DEFINER SET search_path` frequent et migrations recentes de correction.
- Les parcours publics de recherche et panier rendent sans overflow horizontal en desktop et mobile.

P0 - Bloquants avant niveau Uber Eats

1. Webhook Stripe: idempotence non atomique et readiness scale en echec

Evidence:

- `supabase/functions/stripe-webhook/index.ts:532-550` verifie d'abord l'existence de `stripe_webhook_events`, puis fait un `insert` sans inspecter l'erreur.
- `supabase/migrations/20260412211629_stripe_webhook_idempotency.sql:7-12` declare `event_id` en primary key.
- `pnpm run scale:readiness` echoue: "Stripe webhook verifies signatures before side effects".

Risque: en concurrence, deux webhooks Stripe identiques peuvent passer le `maybeSingle()` avant qu'un insert ne soit visible. L'insert concurrent peut echouer sur contrainte unique, mais l'erreur n'est pas lue; le traitement continue et peut produire des side effects dupliques. Pour une plateforme food, c'est critique: confirmation, notification, facturation, reconciliation et refunds doivent etre strictement idempotents.

Correction a faire:

1. Remplacer le pattern select-then-insert par une acquisition atomique:
 - insert `stripe_webhook_events` avant tout side effect;
 - si erreur 23505, retourner `{ duplicate: true }`;
 - si autre erreur, retourner 500 et ne pas traiter;
 - idealement via RPC transactionnelle `claim_stripe_webhook_event(event_id, type, livemode)`.
2. Ajouter un test de contrat qui verifie que l'erreur d'insert est inspectee.
3. Ajuster `scripts/scale-readiness-check.mjs` pour reconnaitre explicitement la verification de signature et l'acquisition atomique.
4. Rejouer `pnpm run scale:readiness` jusqu'a 6/6.

2. `submit-signup-application`: surface publique service-role non liee au JWT

Evidence:

- `supabase/config.toml:142-143`: `submit-signup-application` a `verify_jwt = false`.
- `supabase/functions/submit-signup-application/index.ts:140-180`: la fonction accepte `user_id` et `email` depuis un formulaire multipart.
- `supabase/functions/submit-signup-application/index.ts:216-234`: elle appelle `admin_submit_signup_application` avec un client `admin/service-role`.
- Le gate actuel: UUID, email, Turnstile si configure, rate limits IP/user/global, puis `auth.admin.getUserById(userId)`.

Risque: un acteur qui connait ou obtient `user_id + email` peut soumettre ou polluer un dossier d'onboarding pour ce compte. Le captcha est optionnel si le secret n'est pas configure. Pour une plateforme avec restaurateurs/coursiers, ce flux doit etre lie a une session ou a un token d'onboarding signe.

Correction a faire:

5. Passer `verify_jwt = true` ou appeler `authenticateRequest(req, { allowServiceRole: false })`.
6. Imposer `actor.user.id === userId`.

7. Si l'inscription pre-auth doit rester possible, remplacer `user_id/email` par un token court signe, a usage limite et audite.
8. Rendre Turnstile obligatoire en production pour cette fonction publique.
9. Ajouter tests: refus sans JWT, refus JWT/user_id mismatch, acceptation JWT/user_id match, captcha obligatoire en mode production.

3. Issue GitHub #204: leaked password protection Supabase non prouvee

Evidence:

- Issue ouverte: #204 "Audit: activer la protection Supabase contre les mots de passe compromis".
- Les commentaires existants indiquent que la protection reste a verifier/activer cote Supabase Auth.
- Le connecteur Supabase n'a pas acces au projet TOK `wwcrtyoueexyxxkikaos`; il retourne permission denied pour tables, migrations, fonctions et extensions.
- `pnpm run supabase:doctor:prod` verifie le ciblage local, pas l'etat live Auth.
- Documentation officielle Supabase: leaked password protection est disponible sur Pro+ et rejette les mots de passe compromis via HaveIBeenPwned.

Risque: sans verification live, l'issue de securite reste ouverte et la posture Auth n'est pas au niveau attendu.

Correction a faire:

10. Verifier dans Supabase Dashboard Auth ou via Management API que leaked password protection est activee sur `wwcrtyoueexyxxkikaos`.
11. Ajouter une preuve d'audit: capture Dashboard ou sortie API dans un commentaire GitHub.
12. Ajouter `supabase:auth:doctor` ou une checklist release qui verifie explicitement ce parametre.
13. Fermer #204 seulement apres preuve concrete.

4. Derive Supabase local/dev: home appelle une table absente

Evidence navigateur:

- La home locale produit deux 404:
 - `https://rgzqxxtqwmaylrzxlzob.supabase.co/rest/v1/reservation_progressive_offers?...`
- `src/pages/Index.tsx:209-224` appelle `reservation_progressive_offers`.
- `supabase/migrations/20260615070000_progressive_reservation_offers.sql:4-23` cree la table.

Risque: l'environnement utilise par le frontend local/dev n'est pas aligne avec les migrations du repo. En production, le meme type de derive peut cacher ou casser des rails entiers. Uber Eats-level veut zero erreur reseau connue dans la console sur la home.

Correction a faire:

14. Aligner `.env/target` Supabase local avec l'environnement voulu sans exposer les secrets.
15. Appliquer les migrations au projet dev correspondant ou désactiver le rail par feature flag si la table n'existe pas.
16. Ajouter un fallback UI silencieux pour une feature optionnelle si la table manque.
17. Ajouter un test de readiness qui détecte les tables appelées par les pages publiques mais absentes du target.

P1 - Production readiness et securite

5. `release:readiness` echoue en local

Commande: `pnpm run release:readiness`.

Resultat: FAIL pour variables/secrets de production absents localement: Stripe publishable/secret/webhook, cron secret, Resend, email from, URLs publiques, allowlist origins, Firebase service account, Apple Universal Links, Android keystore.

Interpretation: ce n'est pas necessairement une panne production, car les secrets ne doivent pas etre en local. Mais tant que la readiness ne peut pas s'executer dans l'environnement GitHub production avec preuves, on ne peut pas certifier le lancement.

Action: faire tourner `release:readiness` dans le workflow production ou ajouter un mode CI qui verifie les secrets par presence sans les logger.

6. Gouvernance Edge Functions incomplete

Scan:

- 47 repertoires Edge Functions.
- 44 fonctions configurees dans `supabase/config.toml`.
- Manquantes dans `config.toml`:
 - `cancel-pending-order-checkout`
 - `confirm-match-group-authorization`
 - `create-social-post-boost`

Scan `verify_jwt=false`:

- Fonctions publiques ou gateway JWT off avec gates detectes: Stripe signature, scheduler secret, authenticateRequest, HMAC, captcha/rate limit.
- Trois cas restent a formaliser comme publics-by-design: `track-analytics`, `track-sponsored-event`, `submit-signup-application`.

Action: rendre `supabase/config.toml` exhaustif, ajouter un test "chaque fonction directory a une politique declaree", et exiger pour chaque `verify_jwt=false` un gate reconnu.

7. Drift CI/prod runtime

Evidence:

- `.github/workflows/ci.yml:10-13`: Node 24.
- `.github/workflows/deploy-production.yml:17-20`: Node 22, Supabase CLI 2.102.0.
- `package.json:6-8`: Node `>=22`, npm `>=11` alors que le repo est pnpm.
- La documentation projet mentionne Node 22 et Supabase CLI 2.95.2.

Risque: un code peut passer CI Node 24 et diverger en production Node 22. Les versions Supabase CLI doivent etre documentees comme source de verite.

Action: aligner CI et production sur Node 22, ou justifier Node 24 en prod aussi. Mettre a jour docs/AGENTS si Supabase CLI 2.102.0 est intentionnel.

8. Sessions web stockees dans `localStorage`

Evidence:

- `src/integrations/supabase/authStorage.ts:21-30`: storage web via `localStorage`.
- `src/integrations/supabase/client.ts:24-29`: `persistSession: true`, `autoRefreshToken: true`.
- Native utilise `SecureStorage`, ce qui est mieux.

Risque: courant pour une SPA Supabase, mais insuffisant pour des surfaces admin/paiement au niveau enterprise: un XSS expose les refresh/session tokens.

Action:

18. Pour admin/restaurateur: envisager BFF ou cookies HTTPOnly/SameSite, ou au minimum reauth/MFA pour operations critiques.
19. Ajouter CSP plus stricte progressive avec nonces/hashes si possible.
20. Forcer reauth pour changement mot de passe, role, paiement, remboursements, feature flags critiques.

9. Redirections checkout decentralisees

Scan: plusieurs `window.location.href/assign` vers des URLs retournees par Edge Functions: panier, Zero Attente, Tok One, packs, billing, campagnes.

Risque: si une fonction ou reponse est mal contrainte, une redirection externe non attendue devient possible.

Action: centraliser `redirectToTrustedCheckoutUrl(url)` avec allowlist stricte Stripe Checkout/Portal + same-origin, tests de refus.

10. Dependence vulnerable moderee

Commande: `pnpm audit --prod --json`.

Resultat: 1 moderate, `protobufjs <=7.6.2`, via `firebase > @firebase/firestore > @grpc/proto-loader > protobufjs`, CVE-2026-54269 / GHSA-f38q-mgvj-vph7. Correctif recommande: `protobufjs >=7.6.3`.

Action: mettre a jour Firebase ou resolution pnpm controlee si compatible. Pas de high/critical detecte.

P2 - Performance, scale et maintenabilite

11. Bundles trop lourds

Build prod OK, mais Vite signale des chunks > 500 kB:

- `index-...js`: 545.77 kB minifie, 164.91 kB gzip.
- `observability-vendor`: 469.42 kB minifie.
- `charts-vendor`: 383.34 kB.
- `ui-vendor`: 357.52 kB.

Action: code-splitting plus agressif des surfaces admin/dashboard/charts/observability, prefetch conditionnel, defer Sentry/observability, analyser `rollup-plugin-visualizer`.

12. Requetes potentiellement non paginees

Scan heuristique: 144 chaines `.from(...).select(...)` dans admin/dashboard/hooks/lib sans `limit/range/single/maybeSingle` dans les 18 lignes suivantes.

Exemples:

- `src/pages/admin/AdminCatalog.tsx`: `cuisines`, `collections`, `collection_restaurants`, `restaurants`.
- `src/pages/admin/adminComptaShared.ts`: nombreuses lectures `orders`, `reservations`, `ad_campaigns`, `restaurant_invoices`, `payment_transactions`.
- `src/pages/admin/AdminOrdersReservations.tsx`: `orders`, `reservations`, `profiles`.
- `src/pages/dashboard/dashboardFacturesShared.ts`: `factures`, `commandes`, `reservations`.

Action: transformer les listes admin/dashboard en pagination server-side obligatoire, avec index et limites explicites. Les exports CSV/PDF peuvent utiliser RPC paginee ou jobs.

13. Sinks HTML a durcir

Scan:

- `src/components/ui/chart.tsx:70-80`: `dangerouslySetInnerHTML` pour CSS variables chart.
- `src/lib/accountingExports.ts:690`: `document.write` print HTML.
- `src/components/invoices/TokPayableInvoiceDialog.tsx:114`: `document.write` print HTML.
- `src/pages/admin/AdminComptaAi.tsx:94`: `document.write` print HTML.

La plupart des champs observes echappent deja via `escapeHtml`, mais le niveau cible impose une politique zero surprise.

Action: valider/sanitizer les couleurs chart, limiter les keys CSS a un pattern safe, remplacer `document.write` par DOM construction ou template sanitizer centralise.

Audit UX public

Verification locale:

- Dev server: `http://127.0.0.1:5173/`.
- Browser integre: charge les URLs mais garde `#root` vide dans cette session Vite; modules Vite servis en 200. Limite documentee.
- Playwright/Chrome local via runtime Codex: captures et DOM fiables.
- Captures: `outputs/tok-audit-20260616/screenshots-playwright/`.

Resultats:

- `/recherche` desktop/mobile: rendu OK, pas d'overflow horizontal, 21 restaurants visibles, filtres et tri presents.
- `/panier`: etat vide propre avec CTA restaurants.
- `/auth`: email/password + Google/Apple + resend confirmation.
- `/admin`, `/dashboard`, `/courier`: redirection correcte vers `/auth?redirect=...`
- Console home: deux 404 sur `reservation_progressive_offers` en environnement local/dev.

Finding UX 1: intro video bloquante

Evidence:

- `src/App.tsx:365-368`: `MobileLogoIntro` monte globalement avant navbar/routes.
- `src/components/MobileLogoIntro.tsx:185-217`: `overlay fixed inset-0 z-[9999]`, video autoplay, disparition seulement `onEnded/onError`.
- `src/test/mobile-logo-intro.test.tsx:27-48`: tests verrouillent l'overlay plein ecran et autoplay.
- Capture a 3.5s: intro masque totalement l'app desktop/mobile.

- Capture a 9s: desktop montre enfin l'app; mobile aussi, avec recherche visible.

Impact: un utilisateur qui veut commander maintenant attend une animation de marque avant l'action. Uber Eats et les apps transactionnelles privilegient acces immediat a adresse, recherche, restaurants, panier.

Action:

21. Supprimer l'intro globale ou la transformer en animation non bloquante < 800 ms.
22. La montrer une seule fois par version avec skip visible.
23. Ne jamais bloquer `/recherche`, `/panier`, `/auth`, `/commande/:id`, `/dashboard`, `/admin`, `/courier`.
24. Ajuster les tests pour proteger "access first", pas "overlay first".

Finding UX 2: desktop trop hero-first

Apres intro, desktop 1280x720 affiche surtout branding et grand titre. La recherche n'est pas visible dans le premier viewport. Mobile apres intro est meilleur: ville, recherche, CTA "Je veux manger" et "Restaurateur" sont visibles.

Action: desktop doit afficher immediatement:

- adresse/ville;
- recherche cuisine/restaurant;
- CTA commander/reserver;
- ETA/frais ou promesse de disponibilite;
- 3-6 restaurants/offres visibles sans scroll.

Finding UX 3: navigation et labels

Le header contient beaucoup d'entrees: Explorer, Actualites, Anti-gaspi, Ventes flash, Tok One, Plus, Restaurateurs, Connexion, Help. C'est riche, mais la decision principale est dispersee.

Action: simplifier le premier niveau:

- "Commander" / "Reserver" / "Offres" / "Restaurateurs";
- garder Actualites/Tok One/anti-gaspi dans surfaces secondaires ou rails;
- rendre le panier et la position plus evidents.

Finding UX 4: recherche fonctionnelle mais pas encore marketplace-grade

La recherche liste 21 restaurants, filtres cuisine/budget/note/livraison, tri. C'est une bonne base.

Manques pour niveau Uber Eats:

- ETA et frais de livraison visibles et fiables par restaurant.
- Etat ouvert/ferme/precommande ultra clair.

- Distance, minimum de commande, frais, promos et badge sponsorise.
- Resultats personnalises par adresse, historique, disponibilite et SLA.
- Skeleton/loading et empty states plus riches selon cause.
- Carte ou mode proximite pour Geneve.

Ecart fonctionnel avec Uber Eats

Client

TOK couvre deja recherche, restaurant detail, panier, checkout, reservations, anti-gaspi, flash sales, Tok One, points/cadeaux, social feed et tracking. Pour atteindre le niveau Uber Eats:

- Priorite 1: adresse/localisation comme pivot du produit.
- Priorite 2: ETA, frais, disponibilite et promos dans chaque card.
- Priorite 3: tracking temps reel robuste avec carte, phases, support et preuve de livraison.
- Priorite 4: group orders complets: lien partage, deadline, budget par participant, paiement separe/centralise, tracking commun.
- Priorite 5: support post-commande avec remboursement/incident depuis la commande.

Restaurateur

TOK a un dashboard tres large. Le niveau Uber Eats demande plus de profondeur operationnelle:

- Mode tablette commandes avec alerte sonore/visuelle fiable.
- Commandes planifiees avec notification plein ecran, section dediee, remontee en haut de queue au bon moment.
- Accept/deny avec raisons, temps de preparation dynamique, pause restaurant, rupture article en un tap.
- Roles Admin/Manager/Staff equivalents a Uber Eats Manager.
- Analytics operationnelles: online rate, missed orders, cancellations, issue heatmaps par heure, top items, feedback menu.
- POS/API integration: sync menus, stores, order injection, webhooks, multi-location.

Coursier

TOK a home/jobs/earnings/profile et dispatch. Pour rivaliser:

- Upfront fare avant acceptation.
- Navigation integree, instructions pickup/dropoff, preuve livraison.
- Earnings detaillees, tips, cashout/instant payout ou calendrier clair.
- Heatmap demande, zones, disponibilite, support incident in-app.

- SLA de dispatch, retry, timeout, reassignment et audit.

Admin/operations

TOK a deja des modules admin forts. Pour niveau marketplace:

- Centre incident temps reel: paiements, webhooks, commandes bloquee, restaurants offline, coursiers en retard.
- Playbooks d'action avec ownership, SLA, escalation.
- Observabilite metier: taux paiement, taux acceptation, temps prep, no-courier, refund rate, contact support.
- Outils refund/compensation audites et idempotents.
- Dashboard lancement 10k: throughput, erreurs par fonction, queues, latence Supabase, Vercel logs, edge function logs.

GitHub

Open issue:

- #204: leaked password protection Supabase. Statut: ouverte, justifiee, pas fermable sans preuve live.

Open PRs:

- #244: [feature/strict-feature-flag-governance](#), CI validate failure.
- #235: [fix/tok-photo-source-preservation-20260602](#), checks Supabase Preview/GitGuardian OK.
- #234: [codex/resolve-admin-issues-69-94](#), Supabase/Vercel failures.
- #233: [codex/analyser-faiblesses-pour-rivaliser](#), Supabase Preview failure.
- #232: [chatgpt/match-groupes-prepay-ui](#), CI OK mais Vercel preview failures.

Action: ne pas merger de PR qui touche prod tant que #204, Stripe idempotency et Supabase target/migrations ne sont pas verifiees.

Roadmap pour atteindre le niveau vise

Phase 0 - Bloquants confiance, 1 a 3 jours

25. Atomic idempotency Stripe webhook.
26. JWT binding de [submit-signup-application](#).
27. Activer/verifier leaked password protection Supabase.
28. Corriger le 404 [reservation_progressive_offers](#) sur l'environnement utilise.
29. [scale:readiness](#) a 6/6.

Acceptance:

- Tests nouveaux pour concurrency/duplicate Stripe.
- Tests onboarding JWT mismatch.
- Issue #204 commentee avec preuve, puis fermee seulement si active.
- Zero erreur reseau connue sur la home locale.

Phase 1 - Production gate, 1 semaine

30. Aligner Node CI/prod.
31. Rendre `supabase/config.toml` exhaustif.
32. Ajouter gate `verify_jwt=false` automatique.
33. Corriger ou accepter formellement la vulnerability `protobufjs`.
34. Executer `release:readiness` dans l'environnement production GitHub.
35. Centraliser trusted redirects checkout.

Acceptance:

- CI et deploy utilisent runtime documente.
- `pnpm audit --prod --audit-level moderate` ou exception documentee.
- Release readiness green en environnement production.

Phase 2 - UX marketplace, 2 semaines

36. Supprimer ou reduire l'intro bloquante.
37. Desktop home action-first.
38. Cards restaurants avec ETA/frais/promo/ouvert/ferme.
39. Recherche personnalisee par adresse.
40. E2E Playwright public: home -> search -> restaurant -> cart -> checkout mock.

Acceptance:

- LCP cible < 2.5s sur mobile mid-tier.
- Recherche visible sans scroll sur desktop et mobile.
- 0 overflow horizontal.
- Console sans 404/500 sur routes publiques.

Phase 3 - Operations restaurant/courier, 3 a 6 semaines

41. Order tablet mode.
42. Scheduled orders queue type Uber Eats.
43. Pause/store availability/item availability en temps reel.
44. Courier upfront fare, navigation, proof, earnings.

45. Roles merchant Admin/Manager/Staff.

Acceptance:

- Simulations dispatch/restaurant/courier couvrent accept, prep, pickup, delivered, incident.
- Metrics operationnelles visibles et historisees.

Phase 4 - Scale 10k et observabilite, 4 a 8 semaines

46. Pagination obligatoire sur toutes les listes volumineuses.

47. Event/audit ledger pour paiements, commandes, refunds.

48. Dashboards SLO: auth, checkout, webhook, dispatch, notifications.

49. Load tests 10k et budgets Supabase.

50. Playbooks incident et rollback.

Acceptance:

- Aucune requete non paginee sur table volumineuse.
- Alertes metier branchees.
- RTO/RPO et procedures prod documentees.

Commandes executees

Pass:

- `pnpm run lint`
- `pnpm run lint:release`
- `pnpm run test` -> 241 files, 891 tests OK.
- `pnpm run build`
- `pnpm run build:prod`
- `pnpm exec tsc --noEmit`
- `pnpm audit --prod --audit-level high`
- `pnpm run supabase:doctor:prod`

Fail/attention:

- `pnpm run release:readiness`: FAIL local car secrets/URLs/config mobile production absents.
- `pnpm run scale:readiness`: FAIL 5/6, Stripe webhook signature/side-effect gate.
- `pnpm audit --prod --json`: 1 moderate `protobufjs`.

Definition de "parfait" pour TOK

TOK peut etre considere au niveau Uber Eats pour Geneve quand:

- Aucun PO ouvert.
- `lint`, `test`, `build:prod`, `typecheck`, `release:readiness`, `scale:readiness` passent dans l'environnement cible.
- Supabase live confirme leaked password protection, RLS advisor sans high risk, migrations appliquees au bon projet.
- Checkout/webhook/refund/order finalization sont idempotents sous concurrence.
- Home/search/cart/order tracking sont action-first, rapides, sans erreurs console.
- Dashboard restaurateur gere commandes en temps reel comme un outil de caisse.
- Courier app donne revenus upfront, navigation, statut, preuve, support.
- Admin a un operations center temps reel avec runbooks et audit.
- Tous les flux money/order/support ont tests unitaires, integration et E2E mockes.

Annexes

Artifacts:

- Browser results: `outputs/tok-audit-20260616/screenshots-playwright/browser-results.json`
- Captures:
 - `home-1280x720.png`
 - `home-after-9s-1280x720.png`
 - `home-mobile-390x844.png`
 - `home-mobile-after-9s-390x844.png`
 - `search-1280x720.png`
 - `search-mobile-390x844.png`
 - `auth-1280x720.png`
 - `cart-1280x720.png`
 - `admin-1280x720.png`
 - `dashboard-1280x720.png`
 - `courier-1280x720.png`

Conclusion: la base TOK est ambitieuse et bien couverte par tests, mais le niveau Uber Eats demande d'abord une fermeture stricte des risques paiement/Auth/Supabase, puis une simplification UX autour de l'action immediate et une profondeur operationnelle restaurant/coursier plus proche d'un outil de production temps reel.